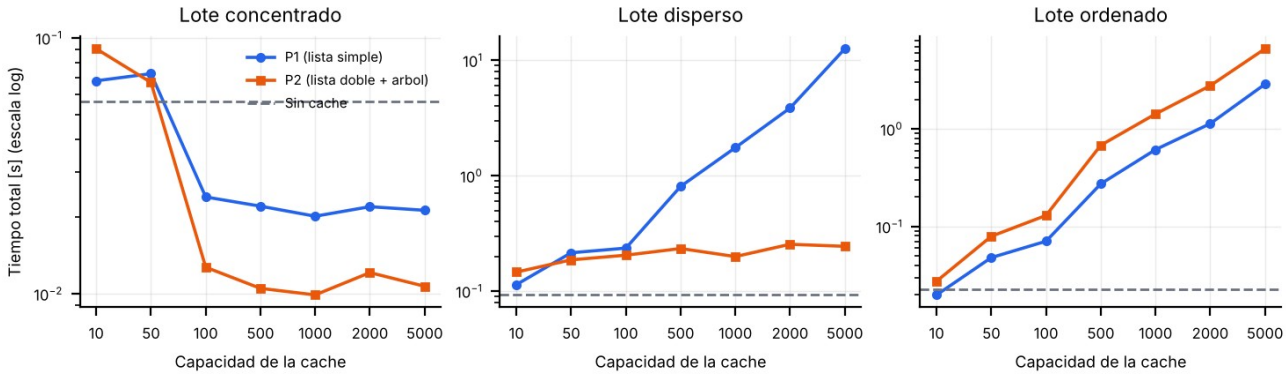
TP LRU Cache – Informe de resultados (Parte 3: simulación Tweener)

Algoritmos y Estructuras de Datos (3640) – UNLaM – 2C 2026 Integrantes: [completar]

1. Condiciones de la simulación

50 000 posteos; cada lote tiene **5 000 usuarios × 20 posteos = 100 000 accesos**; semilla 1; capacidades de caché 10 a 5 000. Los tiempos son la **mediana de 3 corridas** del tiempo total de la simulación; el tiempo por posteo es total/100 000 y por usuario total/5 000 (µs/usuario = 20 × µs/posteo). Las **lecturas al archivo de posteos** son iguales a los fallos (sin caché son 100 000). Medido en Linux (máquina virtual) con gcc -g sin optimizar. Los archivos quedan en la caché del sistema operativo, así que una lectura cuesta ≈ 0,2–0,9 µs, mucho menos que en un disco real; esto achica el beneficio de cualquier caché. Lotes: **concentrado** (ids 0–99), **disperso** (ids al azar en 0–49 999) y **ordenado** (ids no decrecientes en todo el archivo; hay más accesos que posteos, así que cada id aparece unas 2 veces seguidas).

2. Resultados



Caché	Aciertos	Acierto %	Lecturas	P1 total [s]	P1 µs/post.	P2 total [s]	P2 µs/post.	Altura índice
Lote concentrado — sin caché: 0,0566 s total, 0,57 µs/posteo, 11,3 µs/usuario, 100000 lecturas								
10	10.015	10,02	89.985	0,0680	0,68	0,0907	0,91	6
50	49.944	49,94	50.056	0,0725	0,72	0,0670	0,67	7
100	99.900	99,90	100	0,0239	0,24	0,0127	0,13	14
500	99.900	99,90	100	0,0220	0,22	0,0105	0,10	14
1000	99.900	99,90	100	0,0201	0,20	0,0099	0,10	14
2000	99.900	99,90	100	0,0219	0,22	0,0121	0,12	14
5000	99.900	99,90	100	0,0212	0,21	0,0107	0,11	14
Lote disperso — sin caché: 0,0927 s total, 0,93 µs/posteo, 18,5 µs/usuario, 100000 lecturas								
10	28	0,03	99.972	0,1139	1,14	0,1462	1,46	7
50	115	0,12	99.885	0,2149	2,15	0,1866	1,87	8
100	207	0,21	99.793	0,2358	2,36	0,2047	2,05	10
500	985	0,98	99.015	0,8079	8,08	0,2339	2,34	14
1000	1.954	1,95	98.046	1,75	17,50	0,1992	1,99	16
2000	3.961	3,96	96.039	3,86	38,61	0,2546	2,55	20
5000	9.753	9,75	90.247	12,59	125,86	0,2443	2,44	28
Lote ordenado — sin caché: 0,0225 s total, 0,22 µs/posteo, 4,5 µs/usuario, 100000 lecturas								
10	50.000	50,00	50.000	0,0199	0,20	0,0275	0,28	10
50	50.000	50,00	50.000	0,0480	0,48	0,0793	0,79	50
100	50.000	50,00	50.000	0,0706	0,71	0,1298	1,30	100
500	50.000	50,00	50.000	0,2734	2,73	0,6797	6,80	500
1000	50.000	50,00	50.000	0,6131	6,13	1,42	14,22	1000
2000	50.000	50,00	50.000	1,13	11,32	2,77	27,71	2000
5000	50.000	50,00	50.000	2,87	28,72	6,62	66,17	5000

P1 y P2 dan siempre los mismos aciertos y fallos (política LRU idéntica); solo cambia el tiempo.

3. Preguntas del enunciado

¿En qué situaciones la caché mejora el tiempo y en cuáles no? Mejora cuando hay mucha repetición y la capacidad alcanza para el conjunto de posteos en uso. En el lote concentrado (100 posteos distintos), con capacidad ≥ 100 el acierto es 99,9 % y el tiempo baja de 0,0566 s a 0,020–0,024 s con P1 (≈ 2,4–2,8 veces más rápido) y a 0,010–0,013 s con P2 (≈ 4,5–5,7 veces). Con capacidad menor que ese conjunto el acierto es capacidad/100 (10 % y 50 %) y no compensa: con 10 entradas P1 y P2 tardan 0,068 y 0,091 s, más que sin caché. En el lote disperso casi no hay repetición (0,03 % a 9,8 % de aciertos) y la caché nunca mejora: P2 queda 1,6 a 2,7 veces más lento que sin caché y P1 entre 1,2 y 136 veces. En el lote ordenado hay 50 % de aciertos pero tampoco

mejora: sin caché (0,0225 s) es prácticamente lo más rápido y solo P1 con 10 entradas lo iguala (0,0199 s); los ids crecientes se leen casi secuencialmente, la lectura ya es barata y el costo de mantener la estructura supera lo que se ahorra.

¿A partir de qué capacidad deja de convenir la Parte 1? ¿Por qué? Cada acceso recorre la lista en forma lineal, O(entradas guardadas), con un promedio de ce/2 comparaciones. Conviene mientras ese recorrido cueste menos que la lectura que evita (≈ 1 µs acá, del orden de unos cientos de nodos). Con pocos posteos distintos la lista nunca pasa de 100 nodos y P1 sigue siendo útil con cualquier capacidad (0,020–0,024 s). Con poca repetición la lista se llena y el tiempo crece con la capacidad, al menos en forma lineal: 0,24 s (100), 0,81 s (500), 1,75 s (1 000), 3,86 s (2 000) y 12,6 s (5 000). En nuestras mediciones deja de convenir **desde unas 500 entradas**: con 500 P1 es 8,7 veces más lento que sin caché y 3,5 veces más lento que P2; con poca reutilización ya no paga desde 10–100. Con lecturas de disco real (más caras) ese umbral subiría.

¿Qué pasa con el árbol índice en el lote ordenado y cómo afecta? Cada id nuevo es mayor que todos los del árbol, entonces siempre se inserta como hijo derecho y el árbol se convierte en una lista: **la altura es igual a la capacidad** (10, 50, 100, …, 5 000), contra 6–14 en el concentrado y 7–28 en el disperso (≈ $\log_2 n$). Buscar o insertar pasa de O(log n) a O(n) porque la clave más reciente queda en el nodo más profundo; el índice pierde su ventaja y P2 hace el mismo recorrido lineal que P1, con más costo por nodo (árbol → nodo de lista → dato). Resultado: P2 es más lento que P1 con todas las capacidades (con 50, 0,079 contra 0,048 s) y ≈ 2,3 veces más lento con 5 000 (6,62 contra 2,87 s), unas 290 veces peor que no usar caché (0,0225 s). Un árbol balanceado (AVL) mantendría altura O(log n) también con claves ordenadas.